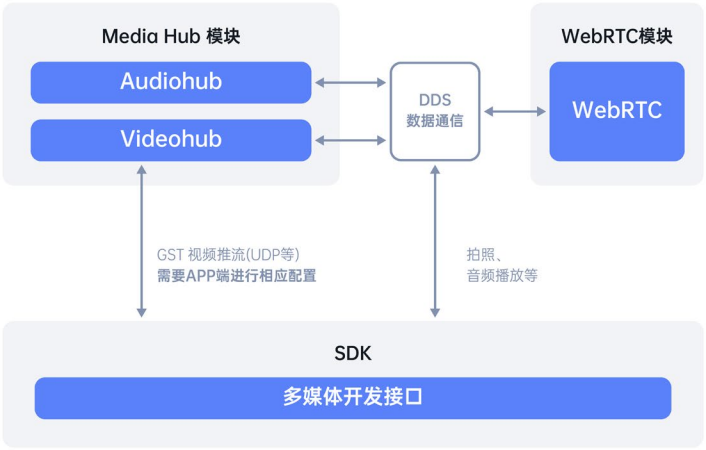
项目 9 多媒体数据获取

1、 多媒体接口

本章节将介绍应用层如何使用多媒体接口，包括获取机器狗摄像头图像数据以及视频流数据等相关应用。

注:

图传推荐使用定向 UDP 接口（也可定制 rtmp 等其他通用接口），不推荐使用 **DDS**（因为 DDS 仅传输 h264 编码后的视频，仍然需要使用者进行视频解码才能最终获得图像），但拍照可以采用 DDS 接口获得 720P 的 JPEG 图像。



2、 JPEG 拍照服务接口

利用该接口，发送方发送拍照请求，随后获得响应，即 JPEG 照片。具体实践是通过 VideoClient 类完成，VideoClient 类是

Video 服务提供的 Client，VideoClient 类可以通过 RPC 方式实现对 Go2 图像的获取。

2.1、 VideoClient 类

接口错误码列表:

类名	构造与析构
VideoClient	VideoClient::~VideoClient()

函数名	GetImageSample
函数原型	int32_t GetImageSample(std::vector<uint8_t>& image_sample)
功能概述	获取照片
参数	**image_sample**：照片内容
返回值	调用成功返回 0，否则返回相关错误码。
备注	

2.2、 案例

本案例展示如何通过 JPEG 拍照服务接口获取图片并保存。

```
#include <unitree/robot/go2/video/video_client.hpp>

#include <iostream>

#include <fstream>

#include <ctime>

int main(int argc, char **argv)

{
```

```
    /*
     * Initilaize ChannelFactory
     */
    if (argc < 2)
    {
        std::cout << "Usage: " << argv[0] << " networkInterface" <<
std::endl;
        exit(-1);
    }
    unitree::robot::ChannelFactory::Instance()->Init(0, argv[1]);
    unitree::robot::go2::VideoClient video_client;
    /*
     * Set request timeout 1.0s
     */
    video_client.SetTimeout(1.0f);
    video_client.Init();
    //Test Api
    std::vector<uint8_t> image_sample;
```

```
    int ret;
    while (true)
    {
        ret = video_client.GetImageSample(image_sample);
        if (ret == 0) {
            time_t rawtime;
            struct tm *timeinfo;
            char buffer[80];
            time(&rawtime);
            timeinfo = localtime(&rawtime);
            strftime(buffer,  sizeof(buffer),  "%Y%m%d%H%M%S.jpg",
timeinfo);
            std::string image_name(buffer);
            std::ofstream image_file(image_name, std::ios::binary);
            if (image_file.is_open()) {
                image_file.write(reinterpret_cast<const
char*>(image_sample.data()), image_sample.size());
                image_file.close();
```

```
                std::cout << "Image saved successfully as " <<
image_name << std::endl;
            } else {
                std::cerr << "Error: Failed to save image." << std::endl;
            }
        }
        sleep(3);
    }
    return 0;
}
```

3、 h264 图传服务接口

在接收端，如需直接查看图传视频流，可以通过在命令行中输入如下指令实现：

```
gst-launch-1.0 udpsrc address=230.1.1.1 port=1720 multicast-
iface=<interface_name> ! queue ! application/x-rtp, media=video,
encoding-name=H264 ! rtph264depay ! h264parse ! avdec_h264 !
videoconvert ! autovideosink
```

其中 230.1.1.1 是多播地址。<interface_name> 是用户主机连接 Go2 的网络接口名称，例如 eth0， 设置成 multicast-iface=eth0 即可。目前只支持 UDP，未来会加入其他图传协议的支持。视频帧率为 15Hz。相机分辨率为 1280*720，水平视角 100 度，垂直视角 56 度。

图传推荐使用 Gst 的定向 UDP 接口。若您打算使用 Gst 来拉流，可参考以下内容，实现在 OpenCV 中使用 Gst 进行拉流。

Gst 支持嵌入 OpenCV 的 VideoCapture 函数中以 Gst pipeline 的方法进行拉流。同时它也支持嵌入 **VideoWriter** 函数中以 Gst pipeline 的方式进行推流。

3.1、 Gst 安装

Ubuntu Linux 环境下安装：

```
sudo apt-get install libgstreamer1.0-dev libgstreamer-plugins-base1.0-dev libgstreamer-plugins-bad1.0-dev gstreamer1.0-plugins-base gstreamer1.0-plugins-good gstreamer1.0-plugins-bad gstreamer1.0-plugins-ugly gstreamer1.0-libav gstreamer1.0-tools gstreamer1.0-x gstreamer1.0-alsa gstreamer1.0-gl gstreamer1.0-gtk3 gstreamer1.0-qt5 gstreamer1.0-pulseaudio
```

说明：

Windows 环境可以参考官方文档：https://gstreamer.freedesktop.org/documentation/installing/on-windows.html?gi-language=c

3.2、 OpenCV 编译

为了实现上述功能，需要在编译 OpenCV 时勾选 **WITH_GSTREAMER** 选项。OpenCV 源码编译过程请参考以下步骤：

① Github 中下载 OpenCV4.1.1

地址: https://github.com/opencv/opencv/tree/4.1.1

② 安装

```
mkdir build

cd build

cmake-gui ..

####此时进入 gui 对话框

####点击 configure

####勾选 with cuda 和 with Gstreamer

####再次点击 configure
```

```
####之后点击 generate

####关闭 gui 对话框

make

sudo make install
```

③OpenCV 中可以通过调用 getBuildInformance 函数查看 OpenCV 的编译情况，具体如下所示：

```
#include <opencv2/opencv.hpp>

int main(void)

{

   std::cout << cv::getBuildInformation() << std::endl;

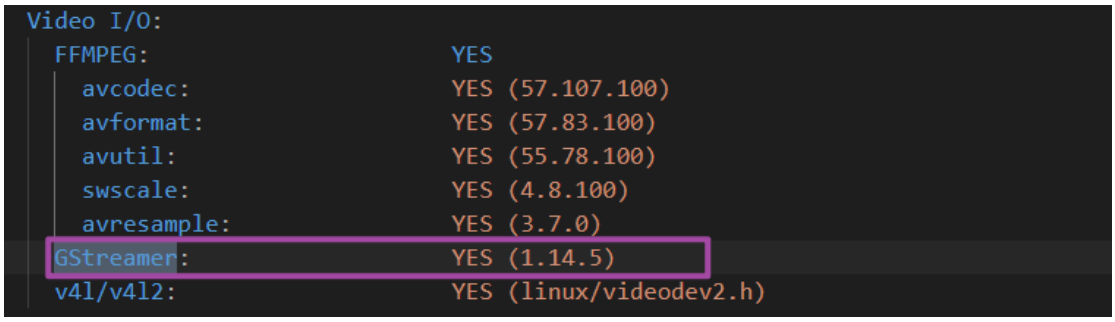
}
```

通过运行上述函数，可以得到如下输出，其中 Video I/O 中可以查看 Gstreamer 选项是否打开。

```
General configuration for OpenCV 4.5.1 =====================================
  Version control:               unknown

  Extra modules:
    Location (extra):
    Version control (extra):     unknown

  Platform:
    Timestamp:                   2022-11-03T02:40:40Z
    Host:                        Linux 5.4.0-42-generic x86_64
    CMake:                       3.22.2
    CMake generator:             Unix Makefiles
    CMake build tool:            /usr/bin/make
    Configuration:               Release
```



3.3、 视频拉流

OpenCV 的 **VideoCapture** 类支持视频文件，图像序列或者摄像头作为输入，获取数据。通过调用如下的 **VideoCapture** 构造函数能实现对 USB 相机数据流的拉取：

```
cv::VideoCapture::VideoCapture(const String & filename,int apiPreference)
```

3.4、 参考案例

若您使用 C++ ，可参考如下代码

```
#include <opencv2/opencv.hpp>
using namespace cv;
#include <iostream>
using namespace std;
```

```
int main()
{
   VideoCapture cap("udpsrc address=230.1.1.1 port=1720 multicast-
iface=<interface_name> ! application/x-rtp, media=video, encoding-
name=H264 ! rtph264depay ! h264parse ! avdec_h264 ! videoconvert !
video/x-raw,width=1280,height=720,format=BGR ! appsink drop=1",
                CAP_GSTREAMER);
   if (!cap.isOpened()) {
            cerr <<"VideoCapture not opened"<<endl;
            exit(-1);
      }
      while (true) {
            Mat frame;
            cap.read(frame);
            imshow("receiver", frame);
            if (waitKey(1) == 27) {
                  break;
            }
```

```
    }
    return 0;
}
```

若您使用 Python，可参考如下代码

```
import cv2
gstreamer_str = "udpsrc address=230.1.1.1 port=1720 multicast-iface=<interface_name> ! application/x-rtp, media=video, encoding-name=H264 ! rtph264depay ! h264parse ! avdec_h264 ! videoconvert ! video/x-raw,width=1280,height=720,format=BGR ! appsink drop=1"
cap = cv2.VideoCapture(gstreamer_str, cv2.CAP_GSTREAMER)
while(cap.isOpened()):
    ret, frame = cap.read()
    if ret:
        cv2.imshow("Input via Gstreamer", frame)
        if cv2.waitKey(25) & 0xFF == ord('q'):
            break
    else:
```

```
            break

cap.release()

cv2.destroyAllWindows()
```

4、 项目总结

通过本项目的深入研究，我们获取了宇树 Go2 机器人的图片快照和视频流数据。基于这些数据，我们将进行更深入的视觉应用项目，包括手势识别和人体追踪应用。